

Reinforcement Learning: Model-Free Value Prediction (ESE 3990)

Instructor:
Antonio Loquercio

Fall 2026
University of Pennsylvania

What We Will Cover in the Next Classes

- Solve a *known* MDP
 - Planning by Dynamic Programming
- Given π , evaluate the value function of an *unknown* MDP
 - Model-free prediction
- Optimize the value function/policy of an *unknown* MDP
 - Model-free control

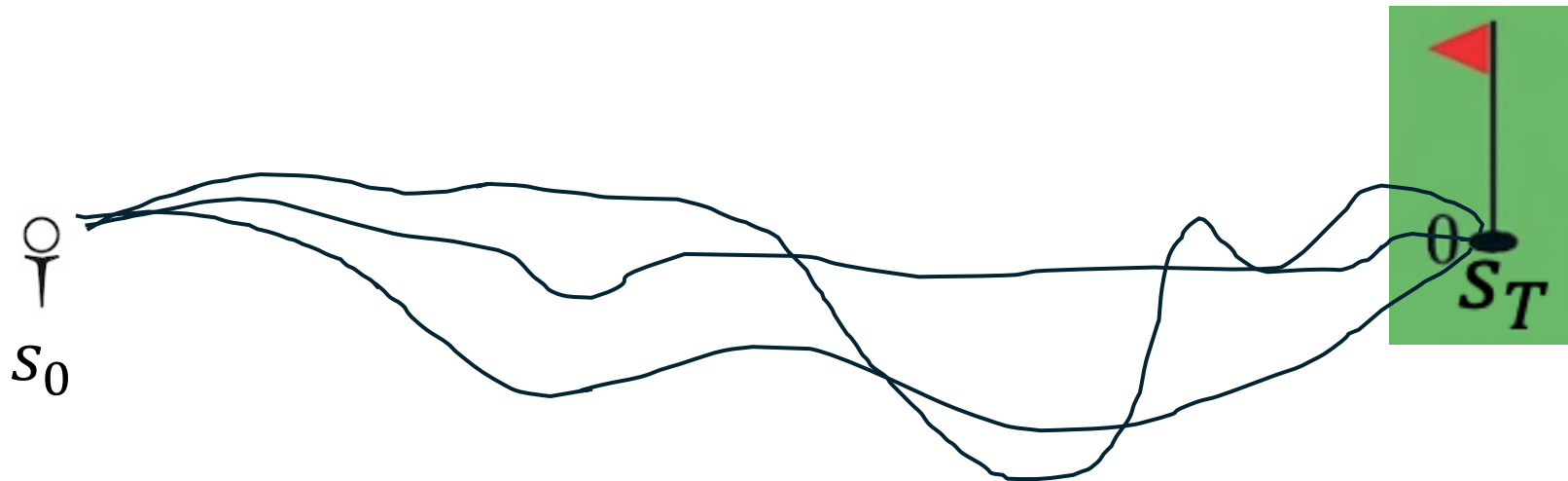
Finding V^π in a novel field with fog

- Idea 1: Trial and Error.
 - Act until you hit the hole.
 - Repeat a few times
 - Average observed returns G_t

$$r(s, a) = -1$$



$$\pi(s) = \text{Putt}$$



Incremental Monte-Carlo Updates

- *Implementation:*
 - For each state s_t with return G_t
 - $N(s_t) \leftarrow N(s_t) + 1$
 - $V(s_t) \leftarrow V(s_t) + \frac{1}{N(s_t)} (G_t - V(s_t))$
- If a problem is *non-stationary* (rules change over time), it is useful to track a running mean, i.e. gradually forget old episodes:
 - $V(s_t) \leftarrow V(s_t) + \alpha (G_t - V(s_t))$
- α is generally called *learning-rate*

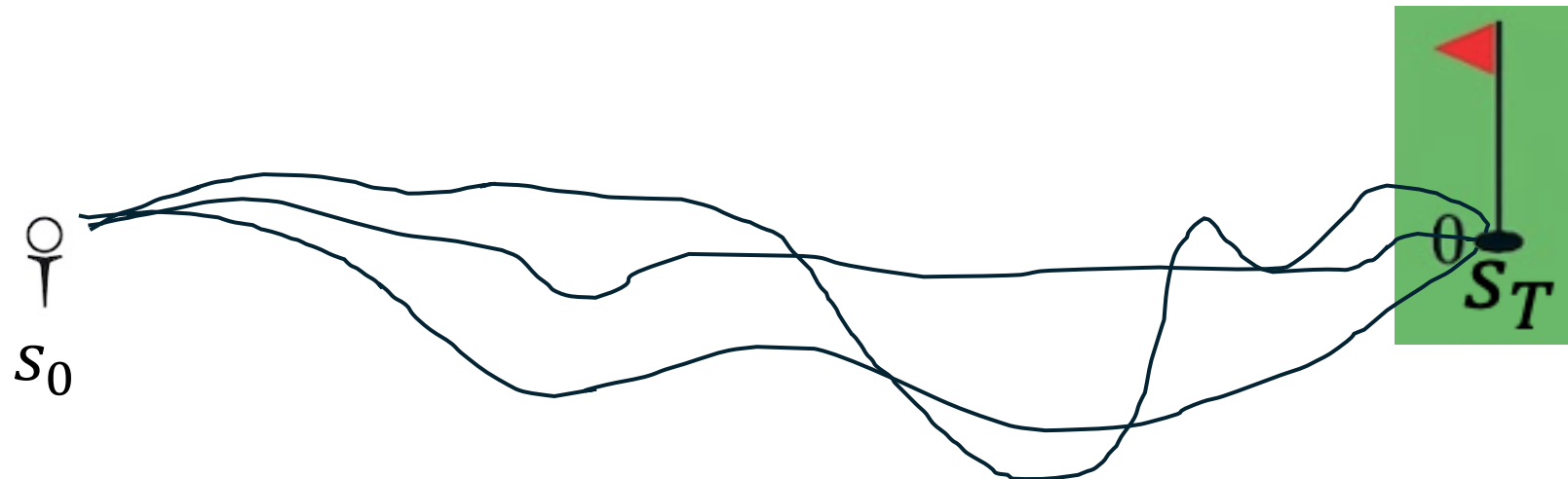
Finding V^π in a novel field with fog

- Quiz:
 - Do I need to repeat the procedure if the environment and policy are deterministic?

$$r(s, a) = -1$$



$$\pi(s) = \text{Putt}$$



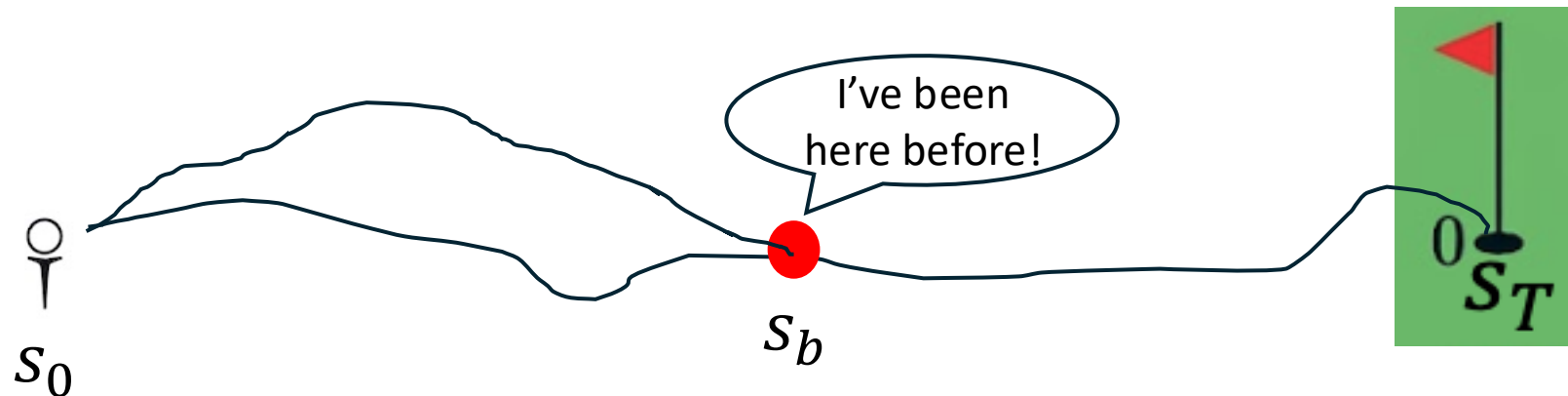
Finding V^π in a novel field with fog

- Idea 2: Reuse prior experience.
- Trial 1: Do the same as MC. Try until you hit.
- In the next trial, I end up in a state s_b visited earlier.

$$r(s, a) = -1$$



$$\pi(s) = \text{Putt}$$



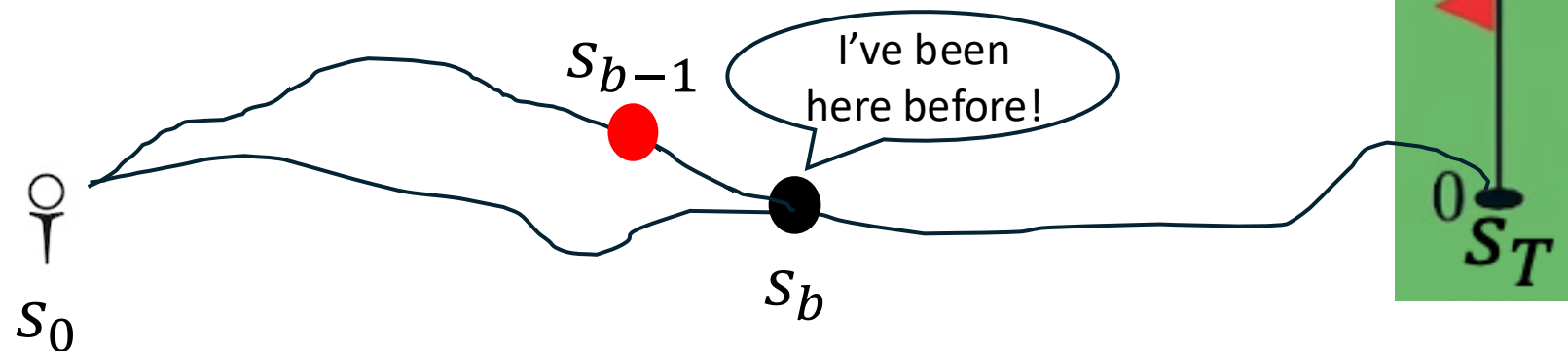
Finding V^π in a novel field with fog

- $\hat{V}^\pi(s_{b-1}) = r_{b-1} + \gamma \hat{V}^\pi(s_b) = -1 + \hat{V}^\pi(s_b)$
- This process is called *bootstrapping*.

$$r(s, a) = -1$$



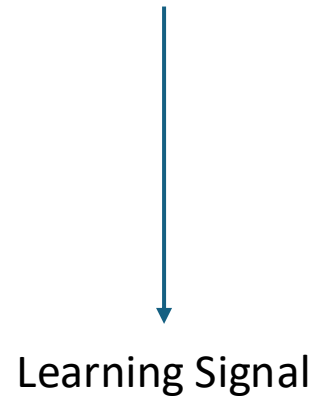
$$\pi(s) = \text{Putt}$$



Monte-Carlo vs Temporal-Difference Learning

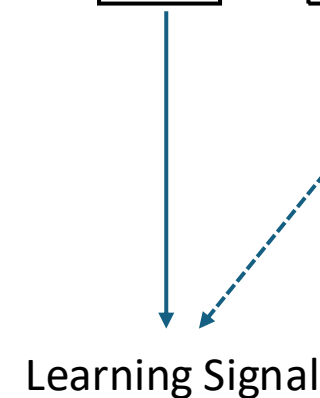
Incremental every-visit MC:

- Update value $V(s_t)$ toward *actual* return G_t
- $V(s_t) \leftarrow V(s_t) + \alpha(G_t - V(s_t))$



TD(0), the simplest TD algorithm:

- Update value $V(s_t)$ toward *estimated* return $r_{t+1} + \gamma V(s_{t+1})$
- $V(s_t) \leftarrow V(s_t) + \alpha(r_{t+1} + \gamma V(s_{t+1}) - V(s_t))$



MC vs TD: Bias/Variance Trade-Off

- MC has high variance, zero bias
- TD has low variance, but some bias

Batch MC / TD

- Assume that we have sampled, up to now, K full episodes:

$$s_1^1, a_1^1, r_2^1, \dots, s_T^1$$

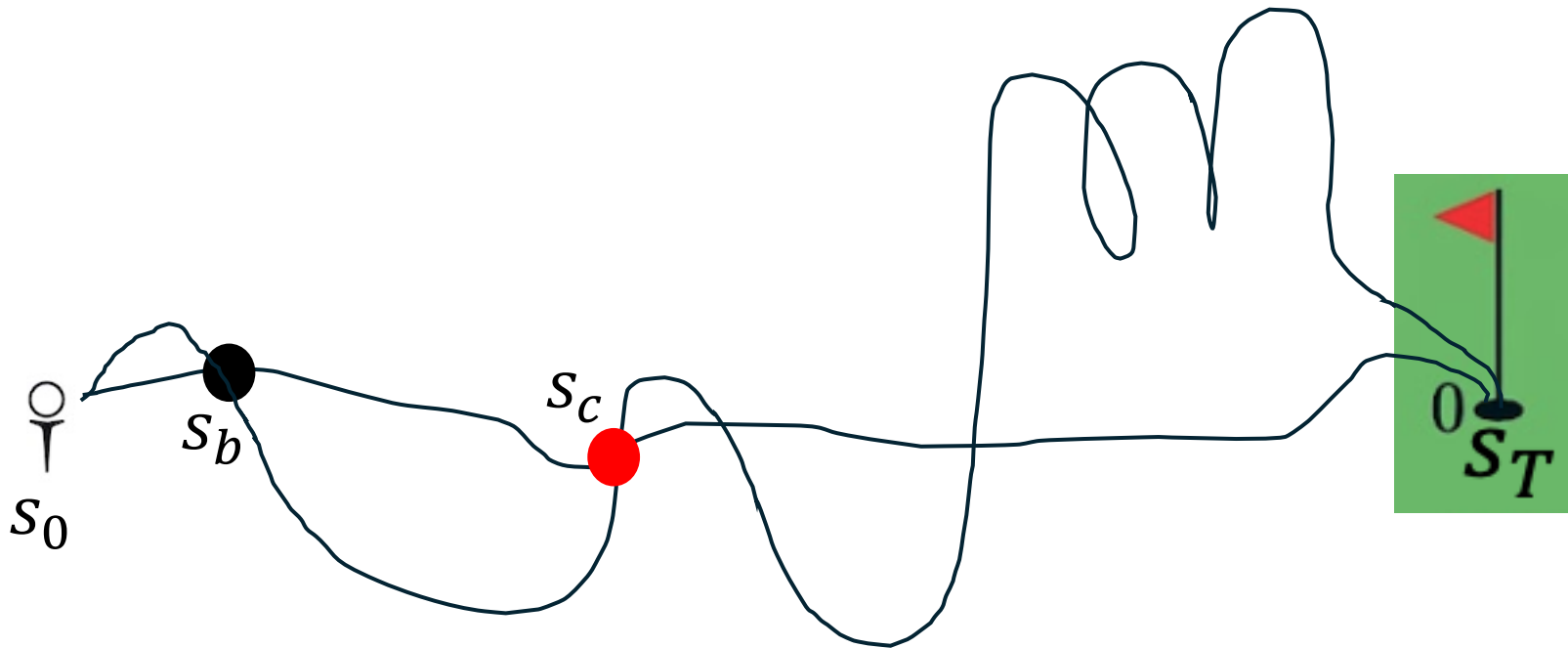
...

$$s_1^K, a_1^K, r_2^K, \dots, s_T^K$$

- *Repeatedly* sample episode $k \in [1, K]$, and apply MC or TD to it.

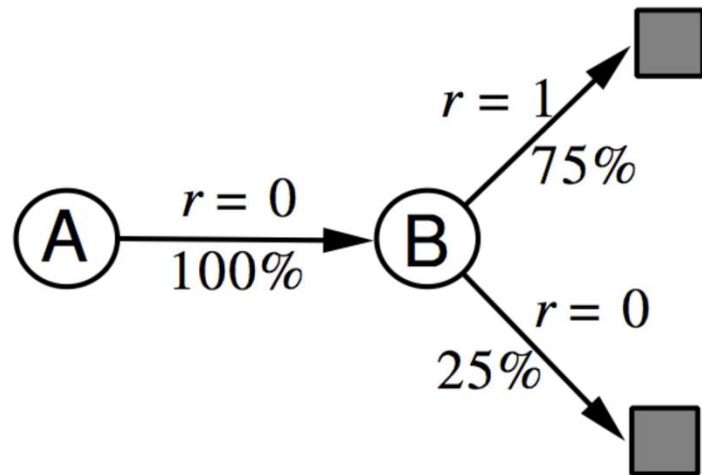
Why is it helpful to revisit old episodes?

- Example: Batch TD(0).
- The experience of the "crazy" trajectory now propagates back to s_b
- *Less bias!*



AB Example:

- Two states A,B; no discounting; 8 episodes of experience.
- Goal: Find $V(A)$ and $V(B)$.
- Let's compare online learning and batch learning.



A, 0, B, 0

B, 1

B, 1

B, 1

B, 1

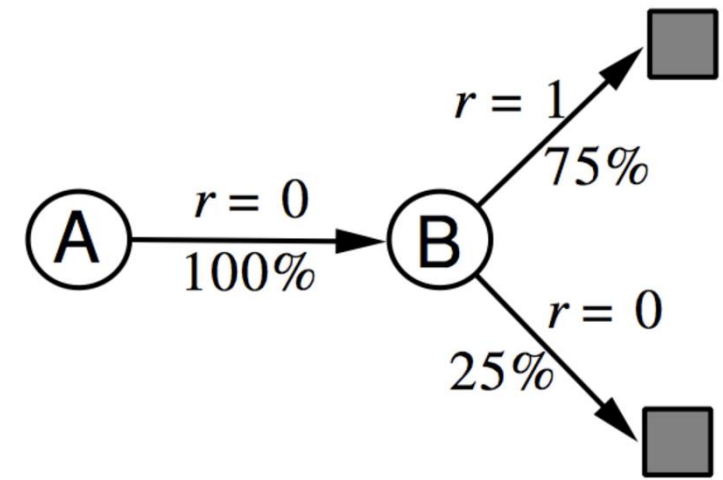
B, 1

B, 1

B, 0

AB Example:

- Always Init $V(A) = 0$ and $V(B) = 0$
- Online learning for MC:
 - At the end: $V(A) = 0, V(B) \approx \frac{6}{8}$
- *Online* Learning for TD(0):
 - At the end: $V(A) = 0, V(B) \approx \frac{6}{8}$
- Batch learning for MC:
 - At the end: $V(A) = 0, V(B) \approx \frac{6}{8}$
- Batch learning for TD(0):
 - At the end: $V(A) \approx \frac{6}{8}, V(B) \approx \frac{6}{8}$



A, 0, B, 0	B, 1
B, 1	B, 1
B, 1	B, 1
B, 1	B, 0

Which one looks like the most correct?

The Connection between TD(0) and MC

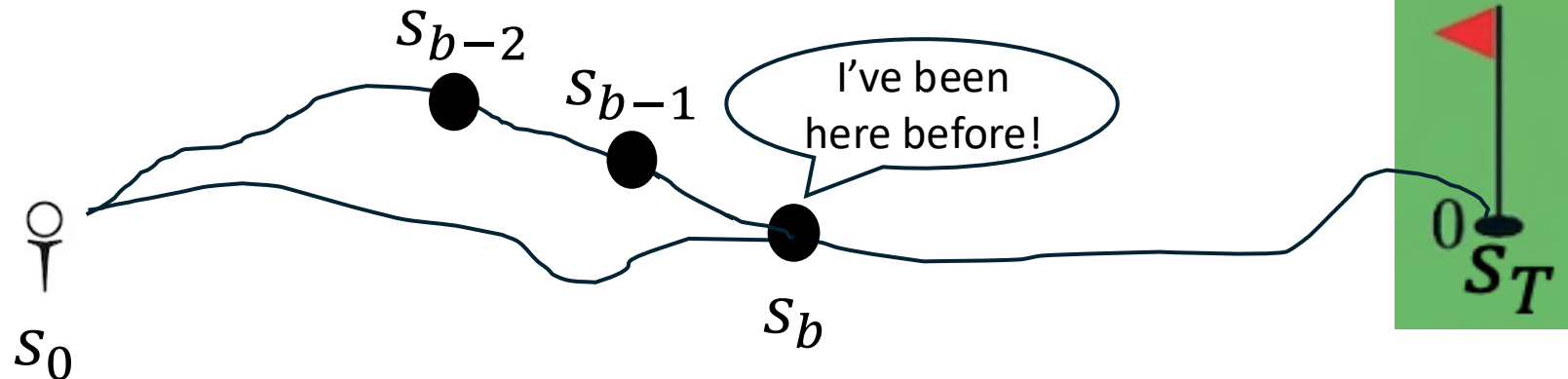
Finding V^π in a novel field with fog

$$r(s, a) = -1$$



$$\pi(s) = \text{Putt}$$

- $\hat{V}^\pi(s_{b-1}) = r_{b-1} + \gamma \hat{V}^\pi(s_b) = -1 + \hat{V}^\pi(s_b)$
- Can we use more than a single step?
- $\hat{V}^\pi(s_{b-2}) = r_{b-1} + \gamma r_{b-2} + \gamma^2 \hat{V}^\pi(s_b) = -2 + \hat{V}^\pi(s_b)$



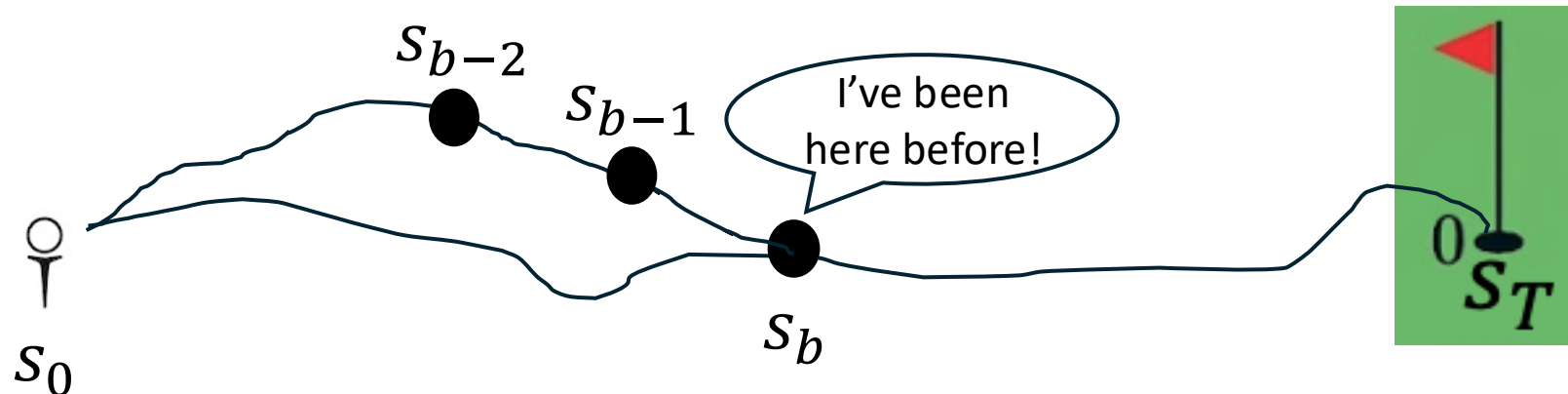
Finding V^π in a novel field with fog

- $\hat{V}^\pi(s_{b-2}) = r_{b-1} + \gamma r_{b-2} + \gamma^2 \hat{V}^\pi(s_b)$
- Do you expect this to have lower or higher *bias* than TD(0)?
- What about *variance*?

$$r(s, a) = -1$$

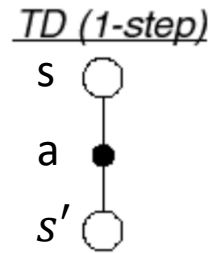


$$\pi(s) = \text{Putt}$$



n-step Prediction

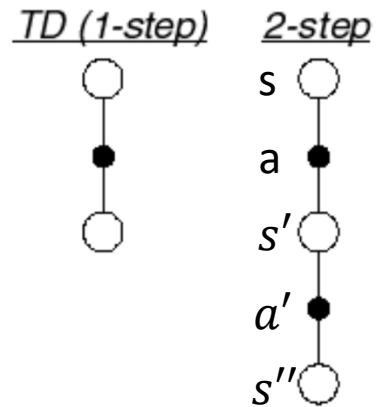
- Up to now. TD learning looks at one step into the future:



$$V(s) = r(s, a) + \gamma V(s')$$

n-step Prediction

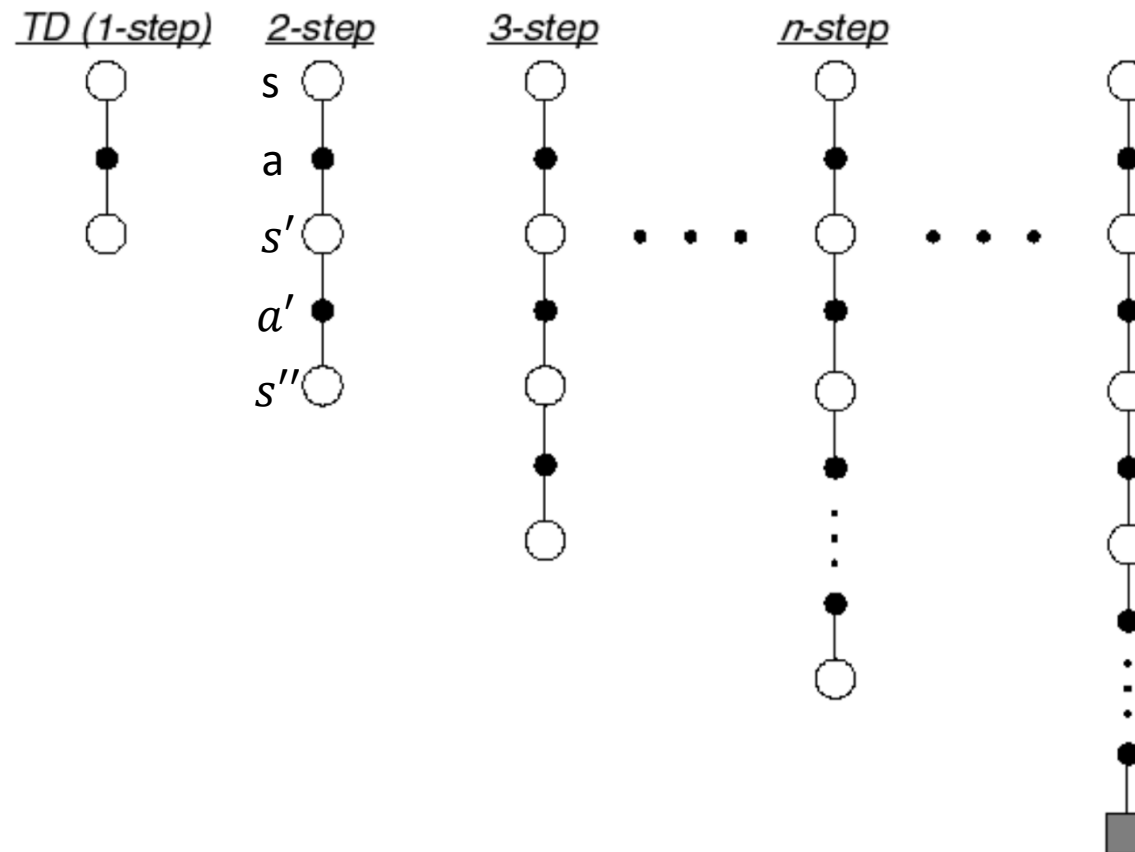
- But why stopping at one step? Why not 2?



$$V(s) = r(s, a) + \gamma r(s', a') + \gamma^2 V(s'')$$

n-step Prediction

- We can use as many steps as the length of the episode.
- How is it called a TD-learning step that uses the entire episode?



N-Step Return

- Consider the following n-step returns for $n = 1, 2, \infty$

$$\begin{array}{ll} n = 1 & (TD) \quad G_t^{(1)} = R_{t+1} + \gamma V(S_{t+1}) \\ n = 2 & G_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2}) \\ \vdots & \vdots \\ n = \infty & (MC) \quad G_t^{(\infty)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T \end{array}$$

- Define the n-step return

$$G_t^{(n)} = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{n-1} r_{t+n} + \gamma^n V(s_{t+n})$$

- N-step temporal-difference learning: Just pick one.

$$V(s_t) \leftarrow V(s_t) + \alpha \left(G_t^{(n)} - V(s_t) \right)$$

N-Step Return

- N-step temporal-difference learning: Just pick one.

$$V(s_t) \leftarrow V(s_t) + \alpha \left(G_t^{(n)} - V(s_t) \right)$$

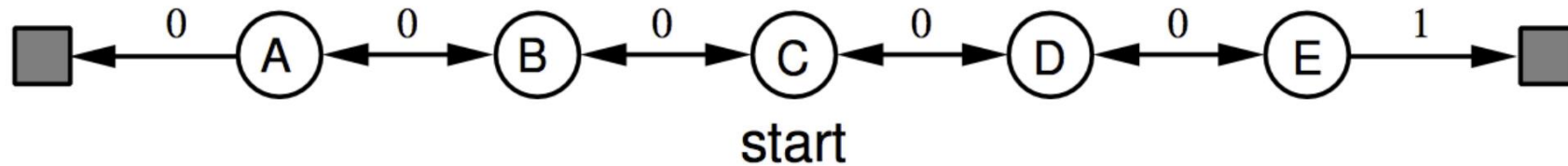
- The lower the number of steps, the smaller the variance.
- The lower the number of steps, the larger the bias.
- We can pick a (n) to trade off bias with variance.

How to Pick a (n)?

- Depends on the problem 😊
- If the environment is Markov and/or highly chaotic, a smaller (n) helps convergence.
- If the environment is non-Markov and/or consistent, a larger (n) helps convergence.

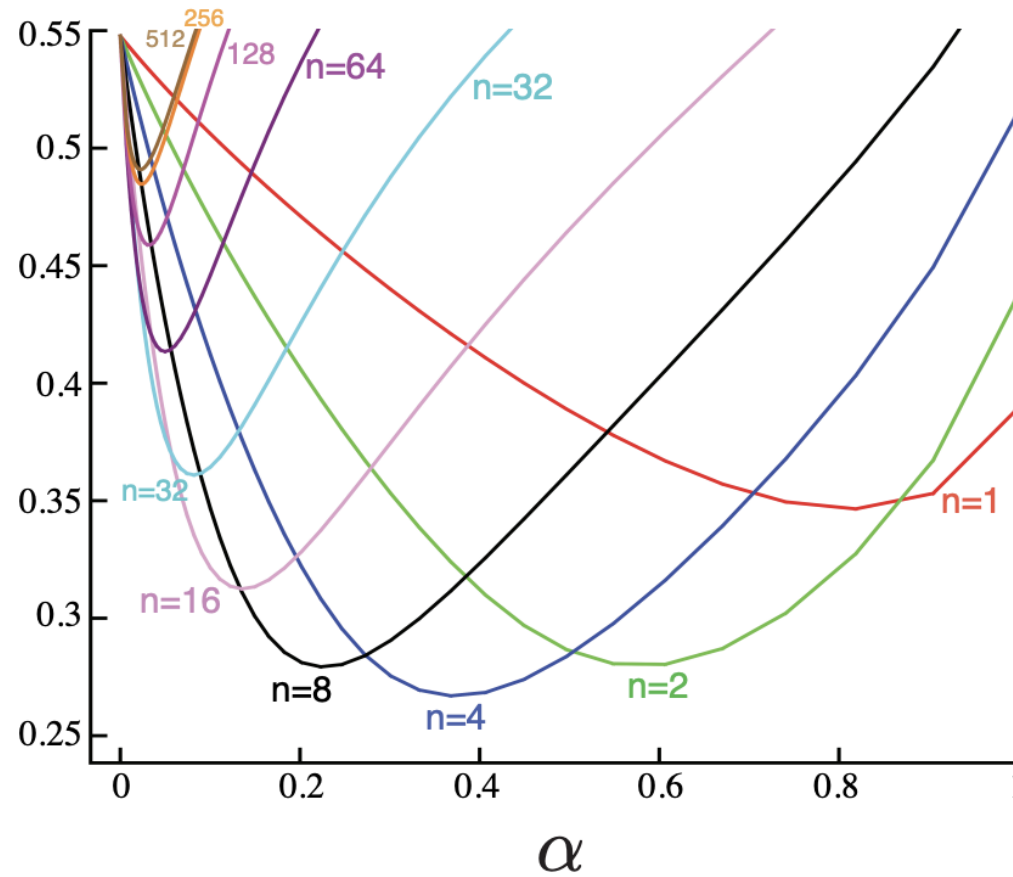
Random Walk Example

- Find $V(s)$ for the following MRP:



Example

Average
RMS error
over 19 states
and first 10
episodes



- MC: $N=512$
- TD(0): $N=1$
- N-step methods perform better than either of the extreme methods

Refresher: Weighted Mean

- Given:
 - An ordered set of numbers $\mathbf{X} = (x_1, x_2, \dots, x_N)$
 - An ordered set of weights $\mathbf{W} = (w_1, w_2, \dots, w_N)$
- The weighted mean of $\mathbf{X}$ with respect to $\mathbf{W}$ is:

$$\mu^w = \frac{\sum_{n=1}^N w_n x_n}{\sum_{n=1}^N w_n}$$

Refresher: Weighted Mean

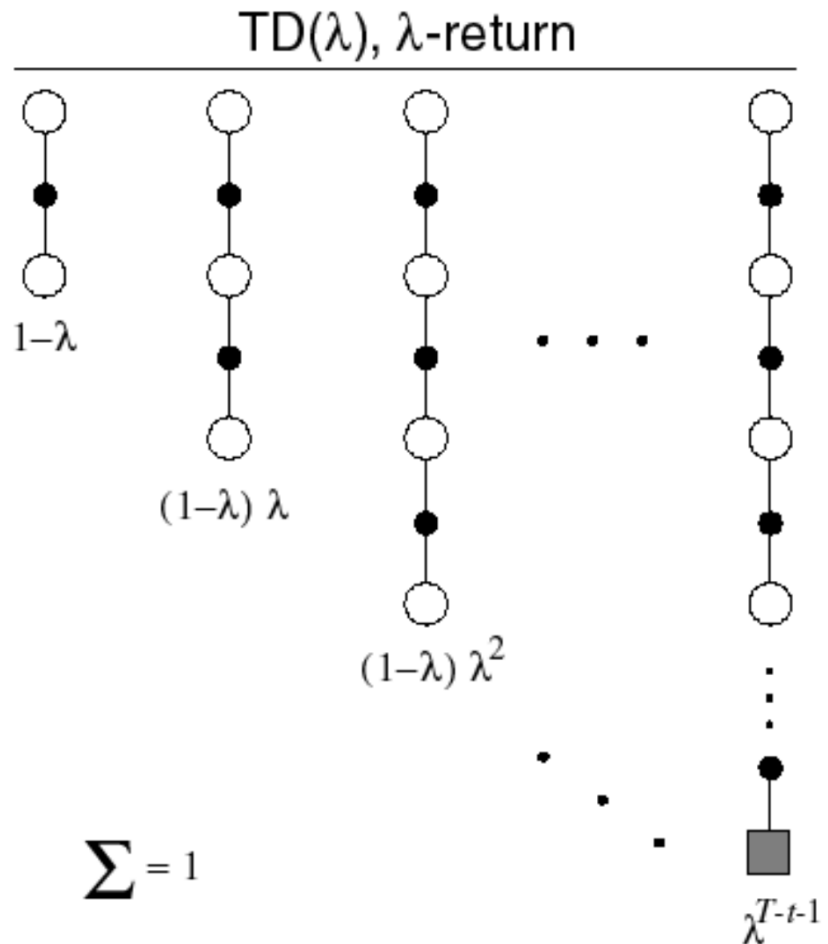
$$\mu^w = \frac{\sum_{n=1}^N w_n x_n}{\sum_{n=1}^N w_n}$$

- Example:
- Example: $W = (1, \lambda, \lambda^2, \dots, \lambda^{N-1})$

$$\mu^w = (1 - \lambda) \sum_{n=1}^N \lambda^{n-1} x_n = \sum_{n=1}^N (1 - \lambda) \lambda^{n-1} x_n$$

- If $\lambda < 1$ and $N \gg 1$

But why pick? Use them all!

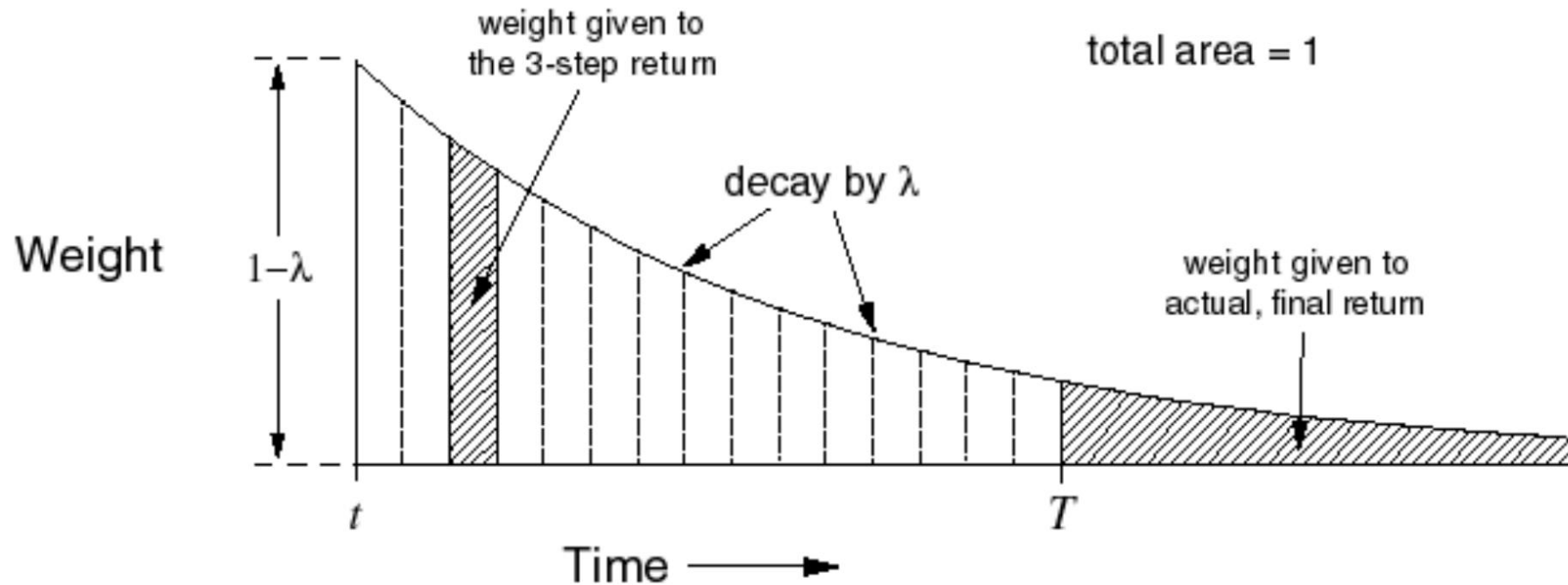


- The λ -return G_t^λ combines *all* n-step returns $G_t^{(n)}$
- Formally: weighted average of $G_t^{(n)}$, using exponentially decaying weight $(1-\lambda)\lambda^{n-1}$:

$$G_t^\lambda = (1-\lambda) \sum_{n=1}^{T-t} \lambda^{n-1} G_t^{(n)}$$

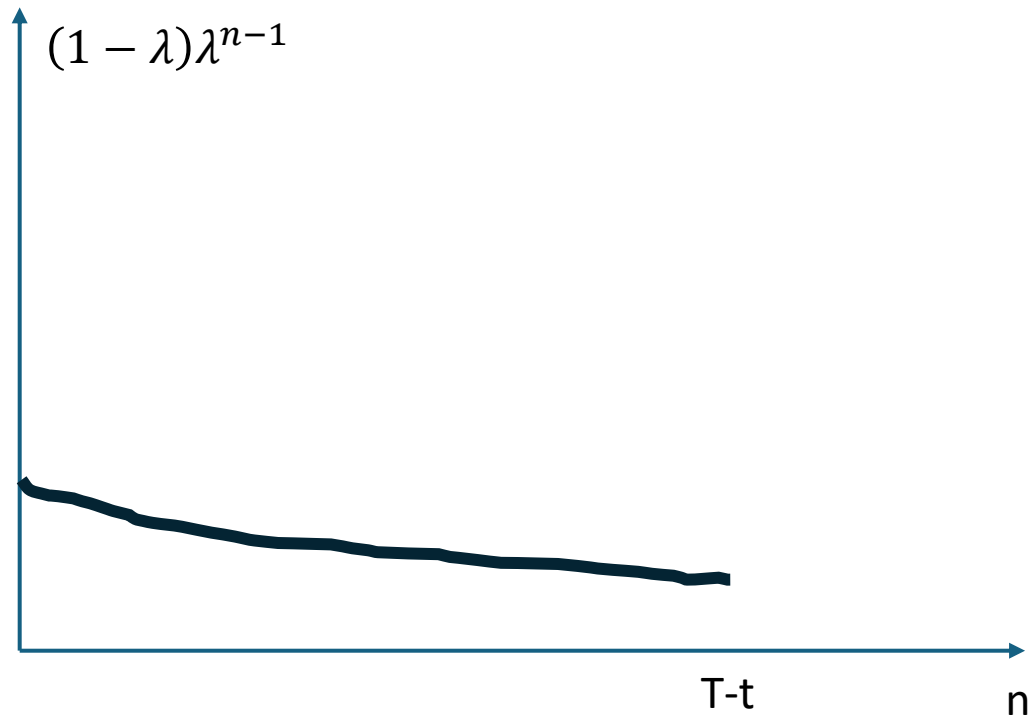
- We call this process $TD(\lambda)$
 $V(s_t) \leftarrow V(s_t) + \alpha(G_t^\lambda - V(s_t))$

$TD(\lambda)$ Weighting Function



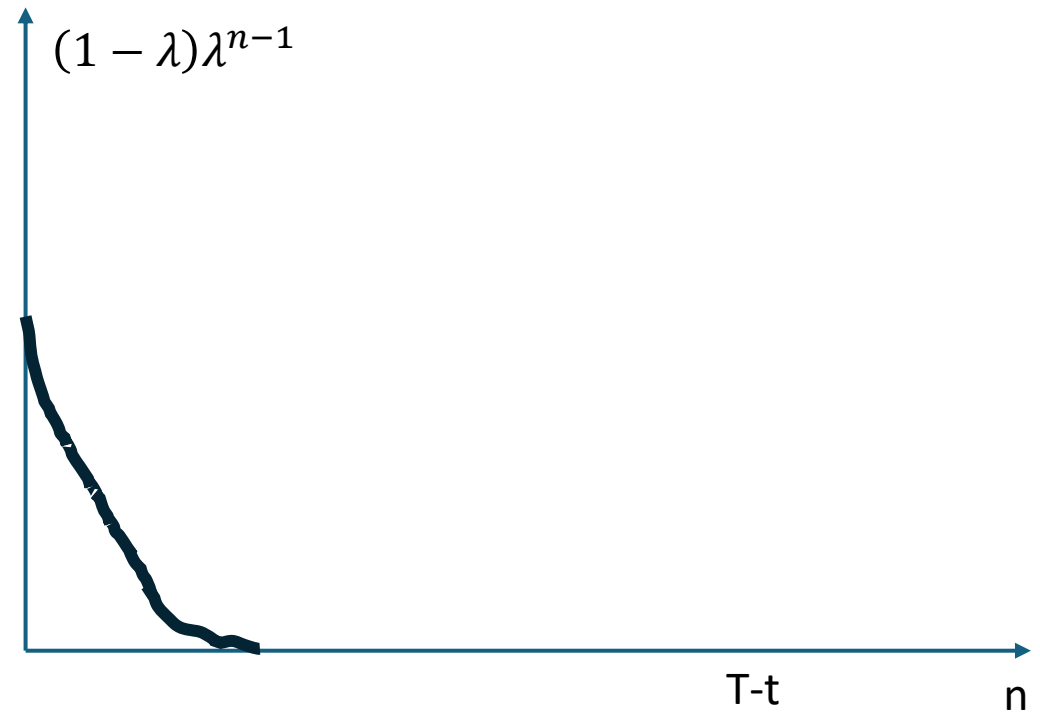
$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{T-t} \lambda^{n-1} G_t^{(n)}$$

$TD(\lambda)$ Weighting Function



$$\lambda \approx 1$$

Far-sighted 



$$\lambda \approx 0$$

Short-sighted 

$TD(\lambda)$ Implementation

- Seems like a nightmare to implement:

1. Compute $G_t^{(n)}$ for all t and n :

$$\forall t, n \quad G_t^{(n)} = r_t + \gamma r_{t+1} + \dots + \gamma^{n-1} V(s_{t+n})$$

2. Average all $G_t^{(n)}$ and compute the $TD(\lambda)$ error:

$$\forall t \quad G_t^\lambda - V(s_t) = (1 - \lambda) \sum_{n=1}^{T-1} \lambda^{n-1} G_t^{(n)} - V(s_t)$$

Telescoping in $TD(\lambda)$ Saves the Day!

- TD errors $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ telescope to λ -error, $G_t^\lambda - V(s_t)$

$$\begin{aligned} G_t^\lambda - V(s_t) &= \sum_{t'=t}^{T-1} (\gamma\lambda)^{t'-t} \delta_{t'} \\ &= \delta_t + \gamma\lambda\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \dots \end{aligned}$$

Telescoping in $TD(\lambda)$ [No exam]

$$\begin{aligned}
 G_t^\lambda - V(S_t) &= -V(S_t) + (1-\lambda)\lambda^0 (R_{t+1} + \gamma V(S_{t+1})) \\
 &\quad + (1-\lambda)\lambda^1 (R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2})) \\
 &\quad + (1-\lambda)\lambda^2 (R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 V(S_{t+3})) \\
 &\quad + \dots \\
 &= -V(S_t) + (\gamma\lambda)^0 (R_{t+1} + \gamma V(S_{t+1}) - \gamma\lambda V(S_{t+1})) \\
 &\quad + (\gamma\lambda)^1 (R_{t+2} + \gamma V(S_{t+2}) - \gamma\lambda V(S_{t+2})) \\
 &\quad + (\gamma\lambda)^2 (R_{t+3} + \gamma V(S_{t+3}) - \gamma\lambda V(S_{t+3})) \\
 &\quad + \dots \\
 &= (\gamma\lambda)^0 (R_{t+1} + \gamma V(S_{t+1}) - V(S_t)) \\
 &\quad + (\gamma\lambda)^1 (R_{t+2} + \gamma V(S_{t+2}) - V(S_{t+1})) \\
 &\quad + (\gamma\lambda)^2 (R_{t+3} + \gamma V(S_{t+3}) - V(S_{t+2})) \\
 &\quad + \dots \\
 &= \delta_t + \gamma\lambda\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \dots
 \end{aligned}$$

$TD(\lambda)$ Implementation

- Very easy with telescoping:

1. Compute all δ_t :

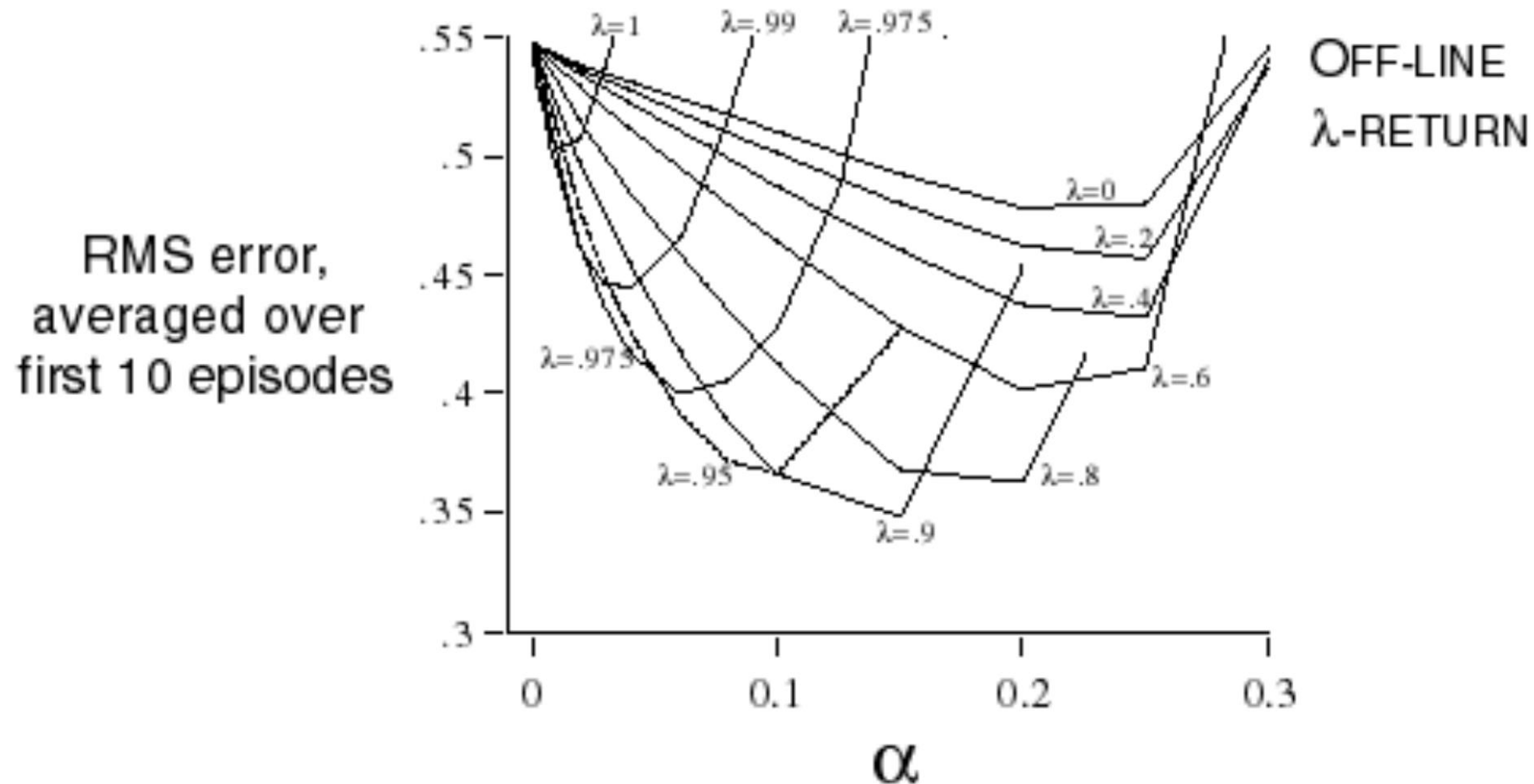
$$\forall t \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

2. Directly compute the $TD(\lambda)$ error:

$$\forall t \quad G_t^\lambda - V(s_t) = \sum_{t'=t}^{T-1} (\gamma\lambda)^{t'-t} \delta_{t'}$$

- No need to recompute for all n!

Forward-view of $TD(\lambda)$ on Large Random Walk



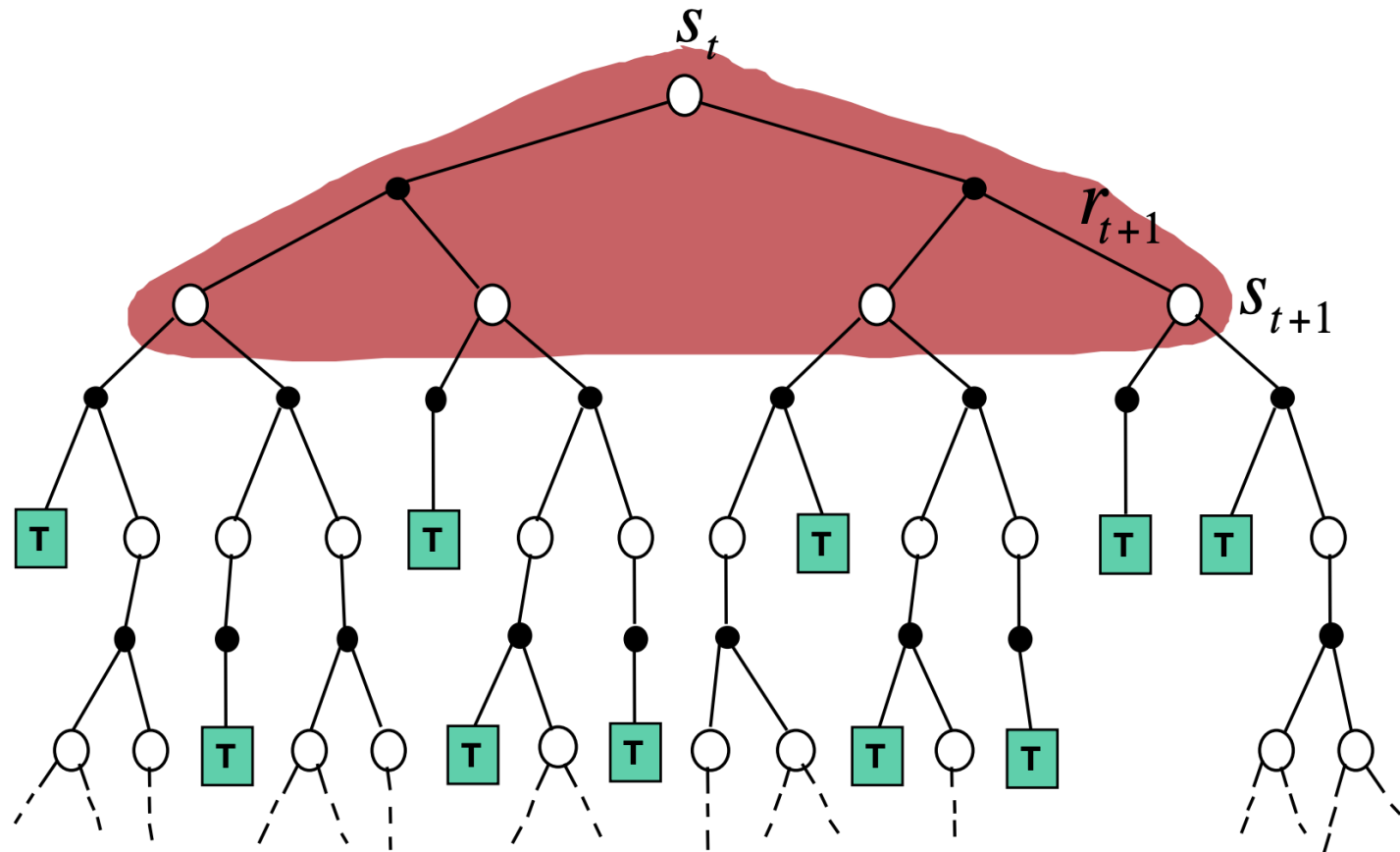
A Unified View of Value Prediction

- We have seen the following methods for value prediction (i.e., finding V^π given π) in the last few classes:
 - Dynamic Programming (Iterative Policy Evaluation)
 - Monte Carlo
 - Temporal-Difference Learning
- All methods share the following characteristics
 - They maintain an approximate value function $V(s)$, $\forall s$
 - They “backup” values according to *actual* or *possible* state trajectories

$$V(s) \leftarrow V(s) + \text{Error}(s)$$

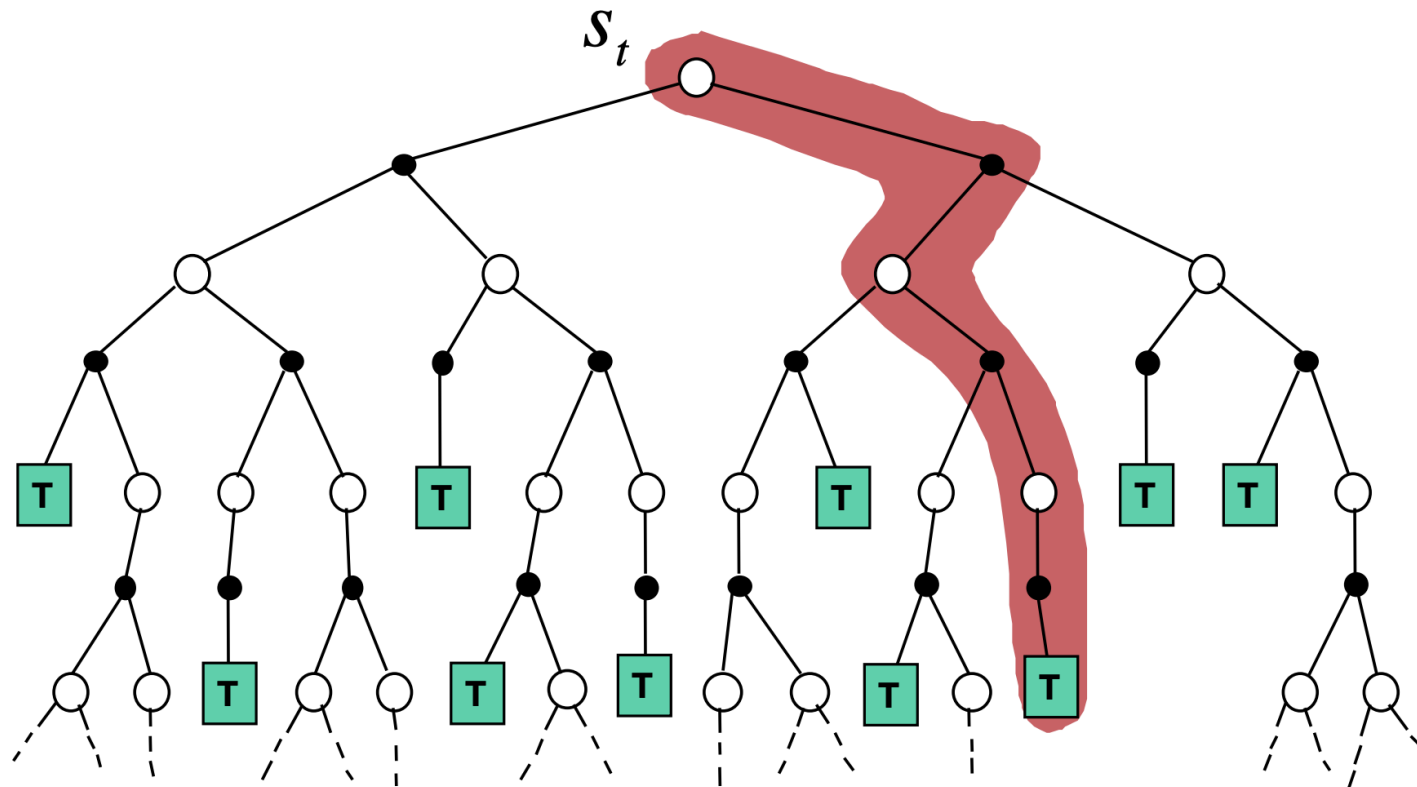
Dynamic Programming Backup

$$V(s_t) \leftarrow \mathbb{E}_{\pi}[r_{t+1} + \gamma V(s_{t+1})]$$



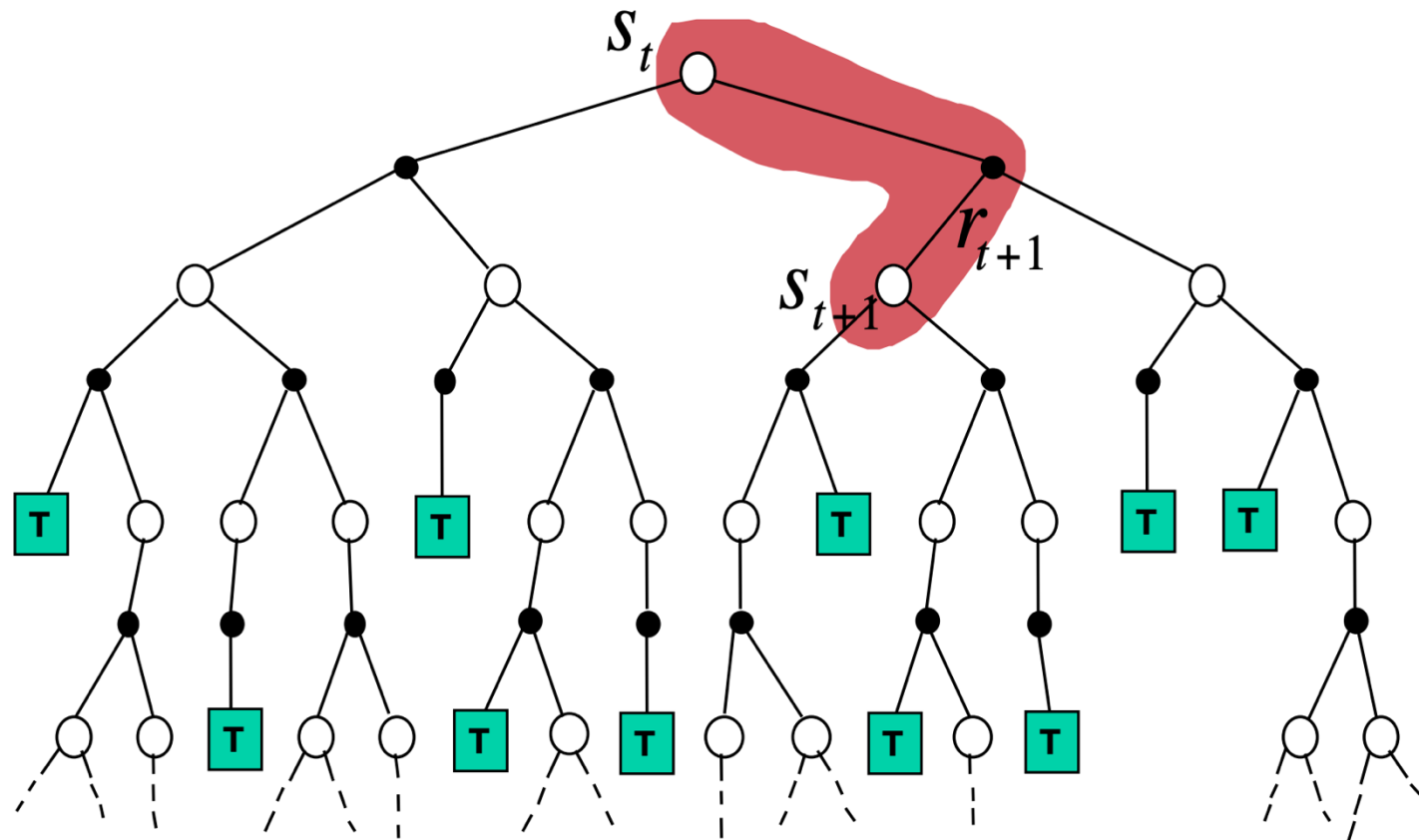
Monte-Carlo Backup

$$V(s_t) \leftarrow V(s_t) + \alpha(G_t - V(s_t))$$

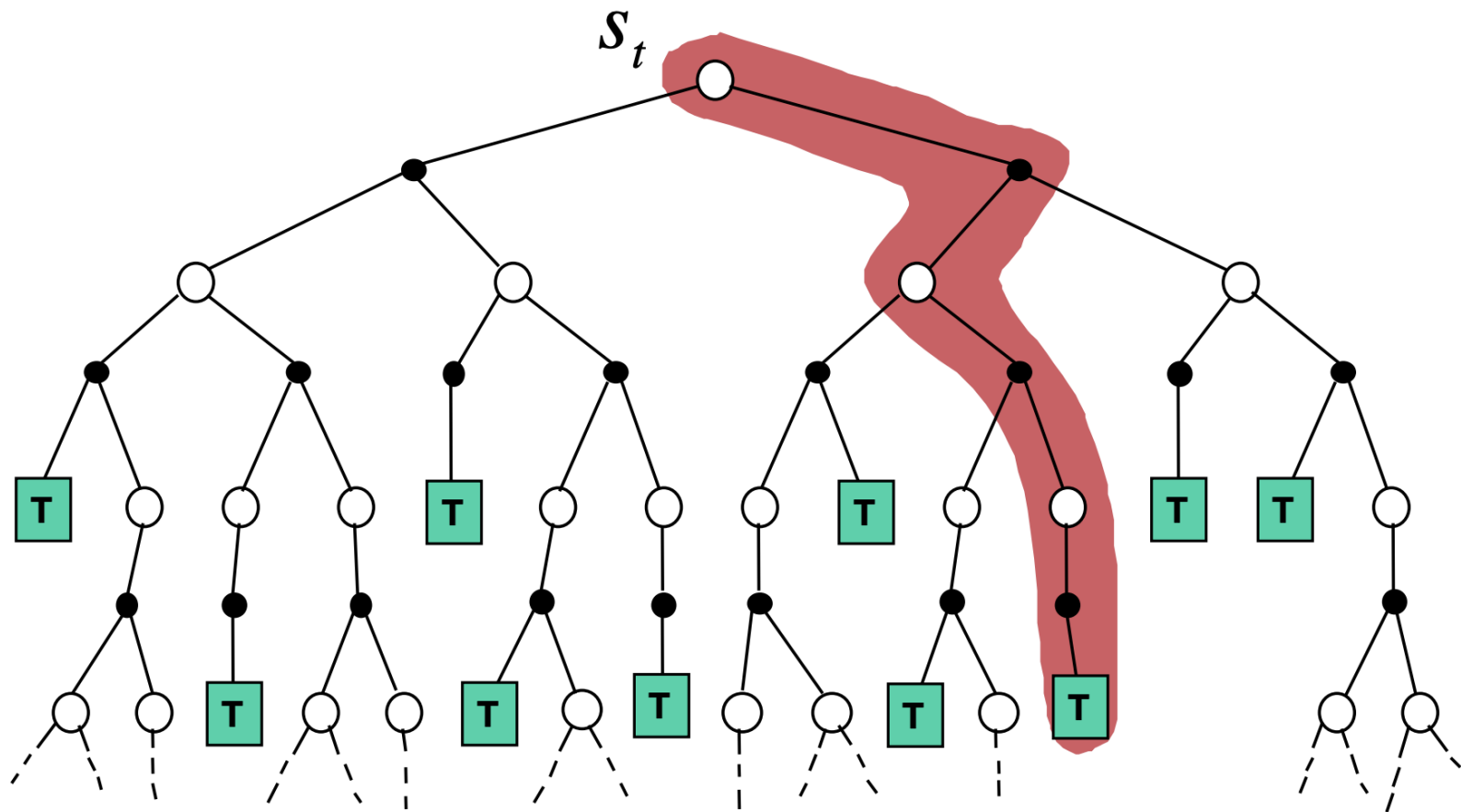


Temporal-Difference Backup

$$V(s_t) \leftarrow V(s_t) + \alpha(r_{t+1} + \gamma V(s_{t+1}) - V(s_t))$$



$TD(\lambda)$ Weighting Function



A Unified View of Value Prediction

What changes between methods?

- **Sampling**: update samples an expectation
 - DP does not sample
 - MC samples
 - TD samples
- **Bootstrapping**: update involves an estimate
 - MC does not bootstrap
 - DP bootstraps
 - TD bootstraps



Baron Munchausen pulls himself up by his bootstraps!